

RCP103 – Rapport de TP

TP02 - Création d'un simulateur de file d'attente

Partie 1

BOURDAYA Amina, BOUTY Gabriel, CLERC Julien

10 mai 2026

Résumé

Ce TP a pour objectif de créer un programme permettant de simuler un système simple de type Client(s)/Server asynchrones. Ce premier document présente l'initialisation du projet, la schématisation du simulateur, et la description de la réalisation des premiers composants.

1 Description du système simulé

Comme évoqué en introduction, ce rapport présente la première phase de réalisation d'un simple simulateur Client(s)/Server :

Un ensemble de N clients ($N \geq 1$), envoient des messages à 1 serveur. Un portail sert d'intermédiaire entre les clients et le serveur. Compte tenu que le serveurs ne peuvent traiter qu'un message à la fois, le portail permet de temporiser les messages en attente de traitement dans une file de taille fixe K .

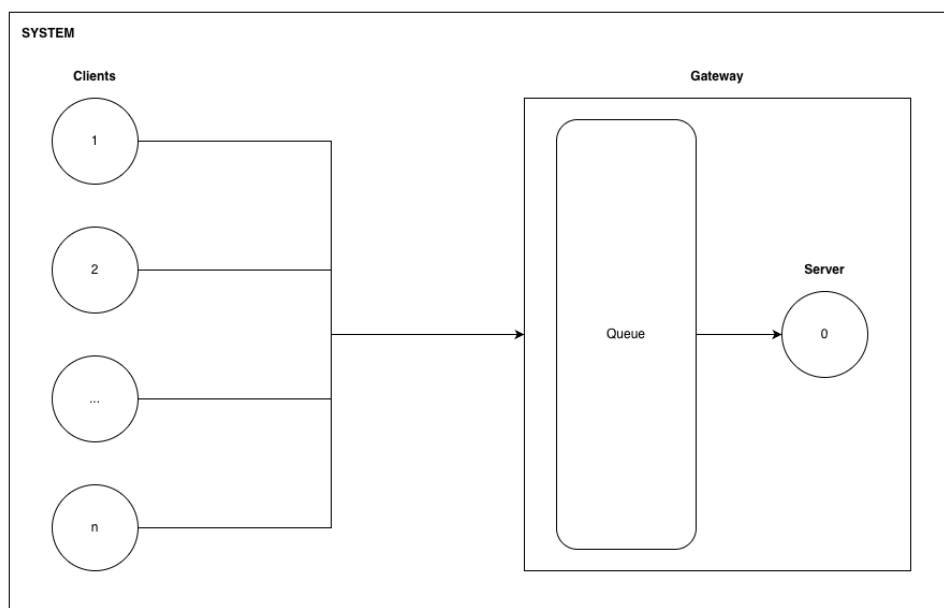


FIGURE 1 – Schéma du système simulé

Dans ce simulateur, les envois de messages suivent un processus de Poisson : le taux d'envois est constant dans le temps, les temps entre deux envois consécutifs suivent une distribution exponentielle. De même, le temps de service suit une loi exponentielle.

Compte tenu du fait que nous n'avons pour l'instant qu'un seul serveur, nous pouvons donc définir cette simulation comme un processus de file d'attente de type M/M/1/K :

- M : Arrivées selon un processus de Poisson (Markovien).
- M : Temps de service selon une distribution exponentielle (Markovien).
- 1 : Nombres de serveurs (ici, 1).
- K : Capacité du système (ici, la taille de la file du portail de capacité K).

La configuration du simulateur est définie comme suit :

- Temps moyen inter-message : configurable
- Temps de transmission : $1t$
- Temps moyen de service : configurable
- Temps total de la simulation : configurable

2 Architecture du simulateur

2.1 Architecture initiale

Afin de simuler les transmissions et traitement de messages entre le ou les *Client* et le *Server* (à travers le *Gateway*), chaque étapes de gestion du *Message* seront encapsulées dans des *Event*. Ces *Event* pourront être, de fait, de 3 types différents :

- SEND : Le message est envoyé par le client.
- RECV : Le message est reçu par le portail.
- DPT : Le message est transmis au serveur (si ce dernier est disponible).

Chacuns de ces *Event* sera timestampé et stocké chronologiquement dans un ordonnanceur (*Scheduler*). Ainsi, en itérant progressivement sur les événements enregistrés dans le *Scheduler*, nous pourrons simuler l'écoulement du temps, en nous concentrant uniquement sur les échantillons temporelles qui nous interesse.

L'ensemble de ces composants seront encapsulés dans un *Engine*, dont la boucle principale viendra interagir avec ces derniers afin de dérouler la simulation.

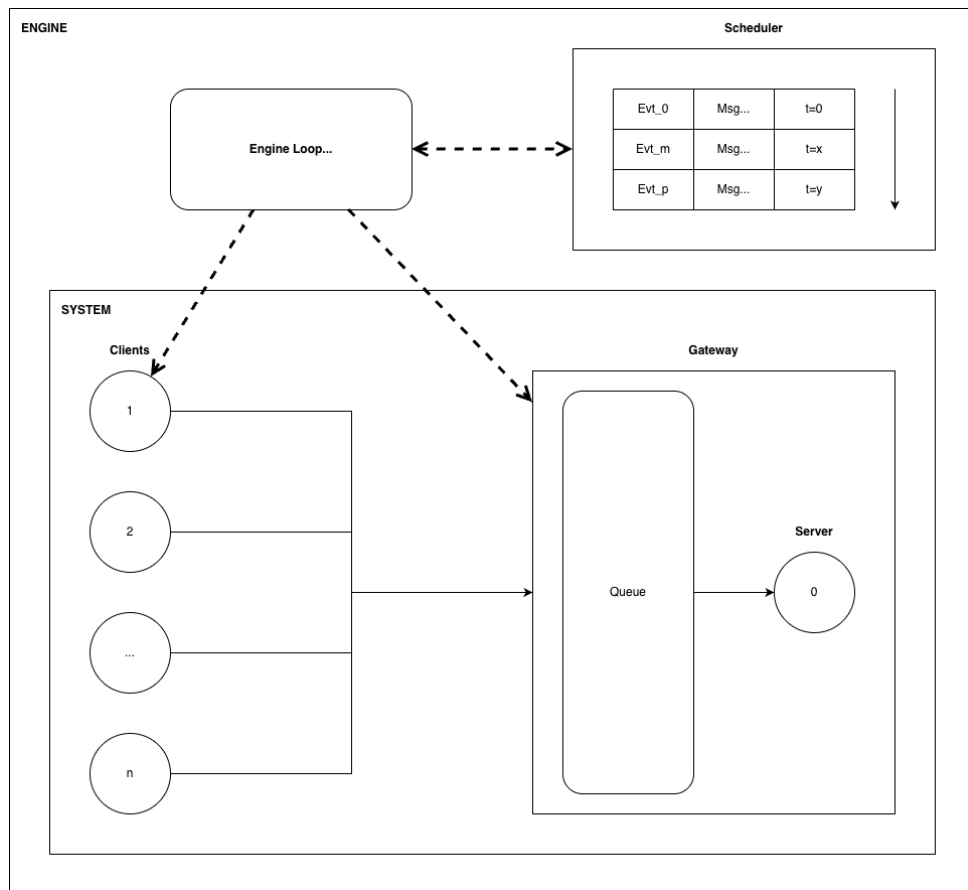


FIGURE 2 – Schéma de principe du simulateur

2.2 Organisation du code source

Le dossier fournit s'organise ainsi :

- *Root* : Le point d'entrée du programme principal (*tp02_main.py*), et les fichiers de configurations de lancement des tests
- Dossier *sources* : Le code applicatif.
- Dossier *tests* : Le code relatifs aux tests unitaires.
- Dossiers *report* et *images* : Le présent rapport les images associées.

3 Créations des premières classes

3.1 Les données : class Message et class Event

3.2 L'ordonnanceur : class Scheduler

3.3 Les fonctions de logs : fonctions Trace

3.4 Le cœur du système : class Engine

3.5 Le point d'entrée : fonction main

4 Tests des premières classes

4.1 Le système de test : Pytest

4.2 Présentation des tests rédigés

4.3 Exécution des tests

5 Conclusion